Formatting Ranges

Document #:	P2286R5
Date:	2022-01-15
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

1 Revision History

Since [P2286R4], several major changes:

- Removed the `d` specifier for delimiters. This paper offers no direct support for changing delimiters (which this paper also in the wording refers to as separators).
- Removed the extra APIs (`retargeted_format_context` and `end_sentry`), and the motivation for their existence.
- Added clearer description of why `range_formatter` is desired and what its exposed API is.
- Updated escaping behavior description with how some other languages do this.
- Added section on container adaptors.
- Added wording.

Since [P2286R3], several major changes:

- Removed the special `pair`/`tuple` parsing for individual elements. This proved complicated and illegible, and led to having to deal with more issues that would make this paper harder to make it for C++23.
- Adding sections on dynamic and static delimiters for ranges. Removing `std::format_join` in their favor.
- Renaming `format_as_debug` to `set_debug_format` (since it's not actually *formatting* anything, it's just setting up)
- Discussing `std::filesystem::path`

Since [P2286R2], several major changes:

- This paper assumes the adoption of [P2418R0], which affects how non-`const`-iterable views are handled. This paper now introduces two concepts (`formattable` and `const_formattable`) instead of just one.
- Extended discussion and functionality for various representations, including how to quote strings properly and how to format associative ranges.

— Introduction of format specifiers of all kinds and discussion of how to make them work more broadly.

— Removed the wording, since the priority is the design.

Since [P2286R1], adding a sketch of wording.

[P2286R0] suggested making all the formatting implementation-defined. Several people reached out to me suggesting in no uncertain terms that this is unacceptable. This revision lays out options for such formatting.

2 Introduction

[LWG3478] addresses the issue of what happens when you split a string and the last character in the string is the delimiter that you are splitting on. One of the things I wanted to look at in research in that issue is: what do *other* languages do here?

For most languages, this is a pretty easy proposition. Do the split, print the results. This is usually only a few lines of code.

Python:

```
print("xyx".split("x"))
```

outputs

```
['', 'y', '']
```

Java (where the obvious thing prints something useless, but there's a non-obvious thing that is useful):

```
import java.util.Arrays;

class Main {
  public static void main(String args[]) {
    System.out.println("xyx".split("x"));
    System.out.println(Arrays.toString("xyx".split("x")));
  }
}
```

outputs

```
[Ljava.lang.String;@76ed5528
[, y]
```

Rust (a couple options, including also another false friend):

```
use itertools::Itertools;

fn main() {
    println!("{:?}", "xyx".split('x'));
    println!("[{}]", "xyx".split('x').format(", "));
```

```
    println!("{:?}", "xyx".split('x').collect::<Vec<_>>());
}
```

outputs

```
Split(SplitInternal { start: 0, end: 3, matcher: CharSearcher { haystack:
        "xyx", finger: 0, finger_back: 3, needle: 'x', utf8_size: 1,
        utf8_encoded: [120, 0, 0, 0] }, allow_trailing_empty: true,
        finished: false })
[, y, ]
["", "y", ""]
```

Kotlin:

```
fun main() {
    println("xyx".split("x"));
}
```

outputs

```
[, y, ]
```

Go:

```
package main
import "fmt"
import "strings"

func main() {
    fmt.Println(strings.Split("xyx", "x"));
}
```

outputs

```
[ y ]
```

JavaScript:

```
console.log('xyx'.split('x'))
```

outputs

```
[ '', 'y', '' ]
```

And so on and so forth. What we see across these languages is that printing the result of split is pretty easy. In most cases, whatever the print mechanism is just works and does something meaningful. In other cases, printing gave me something other than what I wanted but some other easy, provided mechanism for doing so.

Now let’s consider C++.

```
#include <iostream>
#include <string>
#include <ranges>
```

```cpp
#include <format>

int main() {
    // need to predeclare this because we can't split an rvalue string
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    // nope
    std::cout << parts;

    // nope (assuming std::print from P2093)
    std::print("{}", parts);


    std::cout << "[";
    char const* delim = "";
    for (auto part : parts) {
        std::cout << delim;

        // still nope
        std::cout << part;

        // also nope
        std::print("{}", part);

        // this finally works
        std::ranges::copy(part, std::ostream_iterator<char>(std::cout));

        // as does this
        for (char c : part) {
            std::cout << c;
        }
        delim = ", ";
    }
    std::cout << "]\n";
}
```

This took me more time to write than any of the solutions in any of the other languages. Including the Go solution, which contains 100% of all the lines of Go I've written in my life.

Printing is a fairly fundamental and universal mechanism to see what's going on in your program. In the context of ranges, it's probably the most useful way to see and understand what the various range adapters actually do. But none of these things provides an `operator<<` (for `std::cout`) or a formatter specialization (for `format`). And the further problem is that as a user, I can't even do anything about this. I can't just provide an `operator<<` in `namespace std` or a very broad specialization of `formatter` - none of these are program-defined types, so it's just asking for clashes once you start dealing with bigger programs.

The only mechanisms I have at my disposal to print something like this is either

1. nested loops with hand-written delimiter handling (which are tedious and a bad solution), or

2. at least replace the inner-most loop with a `ranges::copy` into an output iterator (which is more differently bad), or
3. Write my own formatting library that I *am* allowed to specialize (which is not only bad but also ridiculous)
4. Use `fmt::format`.

§ 2.1 Implementation Experience

That's right, there's a fourth option for C++ that I haven't shown yet, and that's this:

```
#include <ranges>
#include <string>
#include <fmt/ranges.h>

int main() {
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    fmt::print("{}\n", parts);
    fmt::print("<<{}>>\n", fmt::join(parts, "--"));
}
```

outputting

```
[[], ['y'], []]
<<[]--['y']--[]>>
```

And this is great! It's a single, easy line of code to just print arbitrary ranges (include ranges of ranges).

And, if I want to do something more involved, there's also `fmt::join`, which lets me specify both a format specifier and a delimiter. For instance:

```
std::vector<uint8_t> mac = {0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};
fmt::print("{:02x}\n", fmt::join(mac, ":"));
```

outputs

```
aa:bb:cc:dd:ee:ff
```

`fmt::format` (and `fmt::print`) solves my problem completely. `std::format` does not, and it should.

§ 3 Proposal Considerations

The Ranges Plan for C++23 [P2214R1] listed as one of its top priorities for C++23 as the ability to format all views. Let's go through the issues we need to address in order to get this functionality.

§ 3.1 What types to print?

The standard library is the only library that can provide formatting support for standard library types and other broad classes of types like ranges. In addition to ranges (both the conrete containers like `vector<T>` and the range adaptors like `views::split`), there are several very commonly used types that are currently not printable.

The most common and important such types are `pair` and `tuple` (which ties back into Ranges even more closely once we adopt `views::zip` and `views::enumerate`). `fmt` currently supports printing such types as well:

```
fmt::print("{}\n", std::pair(1, 2));
```

outputs

```
(1, 2)
```

Another common and important set of types are `std::optional<T>` and `std::variant<Ts...>`. `fmt` does not support printing any of the sum types. There is not an obvious representation for them in C++ as there might be in other languages (e.g. in Rust, an `Option<i32>` prints as either `Some(42)` or `None`, which is also the same syntax used to construct them).

However, the point here isn't necessarily to produce the best possible representation (users who have very specific formatting needs will need to write custom code anyway), but rather to provide something useful. And it'd be useful to print these types as well. However, given that `optional` and `variant` are both less closely related to Ranges than `pair` and `tuple` and also have less obvious representation, they are less important.

§ 3.2 Detecting whether a type is formattable

We need to be able to conditionally provide formatters for generic types. `vector<T>` needs to be formattable when `T` is formattable. `pair<T, U>` needs to be formattable when `T` and `U` are formattable. In order to do this, we need to provide a proper `concept` version of the formatter requirements that we already have.

This paper suggests the following:

```
template<class T, class charT>
concept formattable =
    semiregular<formatter<remove_cvref_t<T>, charT>> &&
    requires (formatter<remove_cvref_t<T>, charT> f,
              const formatter<remove_cvref_t<T>, charT> cf,
              T t,
              basic_format_context<fmt-iter-for<charT>, charT> fc,
              basic_format_parse_context<charT> pc) {
        { f.parse(pc) } ->
        same_as<basic_format_parse_context<charT>::iterator>;
        { cf.format(t, fc) } -> same_as<fmt-iter-for<charT>>;
    };
```

The broad shape of this concept is just taking the Formatter requirements and turning them into code. There are a few important things to note though:

— We don't specify what the iterator type is of `format_context` or `wformat_context`, the expectation is that formatters accept any iterator. As such, it is unspecified in the concept *which* iterator will be checked - simply that it is *some* `output_iterator<charT const&>`. Implementations could use `format_context::iterator` and `wformat_context::iterator`, or they could have a bespoke minimal iterator dedicated for concept checking.

— `cf.format(t, fc)` is called on a `const formatter` (see [LWG3636])

— `cf.format(t, fc)` is called specifically on `T`, not a `const T`. Even if the typical formatter specialization will take its object as `const T&`. This is to handle cases like ranges that are not `const`-iterable.

— `formattable<T, char>` and `formattable<T const, char>` could be different, which is important in order to probably know when a range or a `tuple` can be `formattable`.

§ 3.3 What representation?

There are several questions to ask about what the representation should be for printing. I'll go through each kind in turn.

§ 3.3.1 `vector` (and other ranges)

Should `std::vector<int>{1, 2, 3}` be printed as `{1, 2, 3}` or `[1, 2, 3]`? At the time of [P2286R1], `fmt` used `{}`s but changed to use `[]`s for consistency with Python (400b953f).

Even though in C++ we initialize `vectors` (and, generally, other containers as well) with `{}`s while Python's uses `[1, 2, 3]` (and likewise Rust has `vec![1, 2, 3]`), `[]` is typical representationally so seems like the clear best choice here.

§ 3.3.2 `pair` and `tuple`

Should `std::pair<int, int>{4, 5}` be printed as `{4, 5}` or `(4, 5)`? Here, either syntax can claim to be the syntax used to initialize the `pair`/`tuple`. `fmt` has always printed these types with `()`s, and this is also how Python and Rust print such types. As with using `[]` for ranges, `()` seems like the common representation for tuples and so seems like the clear best choice.

§ 3.3.3 `map` and `set` (and other associative containers)

Should `std::map<int, int>{{1, 2}, {3, 4}}` be printed as `[(1, 2), (3, 4)]` (as follows directly from the two previous choices) or as `{1: 2, 3: 4}` (which makes the *association* clearer in the printing)? Both Python and Rust print their associating containers this latter way.

The same question holds for sets as well as maps, it's just a question for whether `std::set<int>{1, 2, 3}` prints as `[1, 2, 3]` (i.e. as any other range of `int`) or `{1, 2, 3}`?

If we print `maps` as any other range of pairs, there's nothing left to do. If we print `maps` as associations, then we additionally have to answer the question of how user-defined associative containers can get printed in the same way. This paper proposes printing the standard library maps as `{1: 2, 3, 4}` and the standard library sets as `{1, 2, 3}`.

3.3.4 `char` and `string` (and other string-like types) in ranges or tuples

Should `pair<char, string>('x', "hello")` print as `(x, hello)` or `('x', "hello")`? Should `pair<char, string>('y', "with\n\"quotes\"")` print as:

```
(y, with
"quotes")
```

or

```
('y', "with\n\"quotes\"")
```

While `char` and `string` are typically printed unquoted, it is quite common to print them quoted when contained in tuples and ranges. This makes it obvious what the actual elements of the range and tuple are even when the string/char contains characters like comma or space. Python, Rust, and `fmt` all do this. Rust escapes internal strings, so prints as `('y', "with\n\"quotes\"")` (the Rust implementation of `Debug` for `str` can be found here which is implemented in terms of `escape_debug_ext`). Following discussion of this paper and this design, Victor Zverovich implemented in this `fmt` as well.

Escaping is the most desirable default behavior, and the specific escaping behavior is described here.

Also, `std::string` isn't the only string-like type: if we decide to print strings quoted, how do users opt in to this behavior for their own string-like types? And `char` and `string` aren't the only types that may desire to have some kind of *debug* format and some kind of regular format, how to differentiate those?

Moreover, it's all well and good to have the default formatting option for a range or tuple of strings to be printing those strings escaped. But what if users want to print a range of strings *unescaped*? I'll get back to this.

3.3.5 `filesystem::path`

We have a paper, [P1636R2], that proposes `formatter` specializations for a different subset of library types: `basic_streambuf`, `bitset`, `complex`, `error_code`, `filesystem::path`, `shared_ptr`, `sub_match`, `thread::id`, and `unique_ptr`. Most of those are neither ranges nor tuples, so that paper doesn't overlap with this one.

Except for one: `filesystem::path`.

During the SG16 discussion of P1636, they took a poll that:

> Poll 1: Recommend removing the filesystem::path formatter from P1636 "Formatters for library types", and specifically disabling filesystem::path formatting in P2286 "Formatting ranges", pending a proposal with specific design for how to format paths properly.

SF	F	A	N	SA
5	5	1	0	0

`filesystem::path` is kind of an interesting range, since it's a range of `path`. As such, checking to see if it would be formattable as this paper currently does would lead to constraint recursion:

```
template <input_range R>
    requires formattable<range_reference_t<R>>
struct formatter<R>
    : range_formatter<range_reference_t<R>>
{ };
```

For `R=filesystem::path`, `range_reference_t<R>` is also `filesystem::path`. Which means that our constraint for `formatter<fs::path>` requires `formattable<fs::path>` Looking at the suggested concept, the first check we will do is to verify that `formatter<fs::path>` is `semiregular`. But we're currently in the process of instantiating `formatter<fs::path>`, it is still incomplete. Hard error.

In order to handle this case properly, we could do what SG16 suggested:

```
template <>
struct formatter<filesystem::path>;
```

But this only handles `std::filesystem::path` and would not handle other ranges-of-self (the obvious example here is `boost::filesystem::path`). So instead, this paper proposes that we first reject ranges-of-self:

```
template <input_range R>
    requires (not same_as<remove_cvref_t<range_reference_t<R>>, R>)
         and formattable<range_reference_t<R>>
struct formatter<R>
    : range_formatter<range_reference_t<R>>
{ };
```

§ 3.3.6 Format Specifiers

One of (but hardly the only) the great selling points of `format` over iostreams is the ability to use specifiers. For instance, from the `fmt` documentation:

```
fmt::format("{:<30}", "left aligned");
// Result: "left aligned                  "
fmt::format("{:>30}", "right aligned");
// Result: "                 right aligned"
fmt::format("{:^30}", "centered");
// Result: "           centered           "
fmt::format("{:*^30}", "centered");  // use '*' as a fill char
// Result: "***********centered***********"
```

Earlier revisions of this paper suggested that formatting ranges and tuples would accept no format specifiers, but there indeed are quite a few things we may want to do here (as by Tomasz Kamiński and Peter Dimov):

— Formatting a range of pairs as a map (the `key: value` syntax rather than the `(key, value)` one)

— Formatting a range of chars as a string (i.e. to print `hello` or `"hello"` rather than `['h', 'e', 'l', 'l', 'o']`)

But these are just providing a specifier for how we format the range itself. How about how we format the elements of the range? Can I conveniently format a range of integers, printing their values as hex? Or as

characters? Or print a range of chrono time points in whatever format I want? That's fairly powerful.

The problem is how do we actually *do that*. After a lengthy discussion with Peter Dimov, Tim Song, and Victor Zverovich, this is what we came up with. I'll start with a table of examples and follow up with a more detailed explanation.

Instead of writing a bunch of examples like `print("{:?}\n", v)`, I'm just displaying the format string in one column (the `"{:?}"` here) and the argument in another (the `v`):

Format String	Contents	Formatted Output
{:}	42	42
{:#x}	42	0x2a
{}	"h\tllo"s	h llo
{:?}	"h\tllo"s	"h\tllo"
{}	vector{"h\tllo"s, "world"s}	["h\tllo", "world"]
{:}	vector{"h\tllo"s, "world"s}	["h\tllo", "world"]
{::}	vector{"h\tllo"s, "world"s}	[h llo, world]
{:*^14}	vector{"he"s, "wo"s}	*["he", "wo"]*
{::*^14}	vector{"he"s, "wo"s}	[******he******, ******wo******]
{}	vector<char>{'H', '\t', 'l', 'l', 'o'}	['H', '\t', 'l', 'l', 'o']
{::}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[H, , l, l, o]
{::c}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[H, , l, l, o]
{::?}	vector<char>{'H', '\t', 'l', 'l', 'o'}	['H', '\t', 'l', 'l', 'o']
{::d}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[72, 9, 108, 108, 111]
{::#x}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[0x48, 0x9, 0x6c, 0x6c, 0x6f]
{:s}	vector<char>{'H', '\t', 'l', 'l', 'o'}	H llo
{:?s}	vector<char>{'H', '\t', 'l', 'l', 'o'}	"H\tllo"
{}	pair{42, "h\tllo"s}	(42, "h\tllo")
{}	vector{pair{42, "h\tllo"s}}	[(42, "h\tllo")]
{:m}	vector{pair{42, "h\tllo"s}}	{42: "h\tllo"}
{:m:}	vector{pair{42, "h\tllo"s}}	{42: h llo}
{}	vector{vector{'a'}, vector{'b', 'c'}}	[['a'], ['b', 'c']]

Format String	Contents	Formatted Output
{::?s}	vector{vector{'a'}, vector{'b', 'c'}}	["a", "bc"]
{:::d}	vector{vector{'a'}, vector{'b', 'c'}}	[[97], [98, 99]]

3.3.7 Explanation of Added Specifiers

3.3.7.1 The debug specifier ?

`char` and `string` and `string_view` will start to support the `?` specifier. This will cause the character/string to be printed as quoted (characters with `'` and strings with `"`) and all characters to be escaped, as described earlier.

This facility will be generated by the formatters for these types providing an addition member function (on top of `parse` and `format`):

```
void set_debug_format();
```

Which other formatting types may conditionally invoke when they parse a `?`. For instance, since the intent is that range formatters print escaped by default, the logic for a simple range formatter that accepts no specifiers might look like this (note that this paper is proposing something more complicated than this, this is just an example):

```
template <typename V>
struct range_formatter {
    std::formatter<V> underlying;

    template <typename ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        // ensure that the format specifier is empty
        if (ctx.begin() != ctx.end() && *ctx.begin() != '}') {
            throw std::format_error("invalid format");
        }

        // ensure that the underlying type can parse an empty specifier
        auto out = underlying.parse(ctx);

        // conditionally format as debug, if the type supports it
        if constexpr (requires { underlying.set_debug_format(); }) {
            underlying.set_debug_format();
        }
        return out;
    }

    template <typename R, typename FormatContext>
        requires
        std::same_as<std::remove_cvref_t<std::ranges::range_reference_t<R>>,
        V>
    constexpr auto format(R&& r, FormatContext& ctx) {
```

```
        auto out = ctx.out();
        *out++ = '[';
        auto first = std::ranges::begin(r);
        auto last = std::ranges::end(r);
        if (first != last) {
            // have to format every element via the underlying formatter
            ctx.advance_to(std::move(out));
            out = underlying.format(*first, ctx);
            for (++first; first != last; ++first) {
                *out++ = ',';
                *out++ = ' ';
                ctx.advance_to(std::move(out));
                out = underlying.format(*first, ctx);
            }
        }
        *out++ = ']';
        return out;
    }
};
```

3.3.7.2 Range specifiers

Range format specifiers come in two kinds: specifiers for the range itself and specifiers for the underlying elements of the range. They must be provided in order: the range specifiers (optionally), then if desired, a colon and then the underlying specifier (optionally).

Some examples:

specifier	meaning
`{}`	No specifiers
`{:}`	No specifiers
`{:<10}`	The whole range formatting is left-aligned, with a width of 10
`{:*^20}`	The whole range formatting is center-aligned, with a width of 20, padded with `*`s
`{:m}`	Apply the `m` specifier to the range (which must be a range of pair or 2-tuple)
`{::d}`	Apply the `d` specifier to each element of the range
`{:?s}`	Apply the `?s` specifier to the range (which must be a range of char)

There are only a few top-level range-specific specifiers proposed:

— `s`: for ranges of char, only: formats the range as a string.

— `?s` for ranges of char, only: same as `s` except will additionally quote and escape the string.

— `m`: for ranges of `pairs` (or `tuples` of size 2) will format as `{k1: v1, k2: v2}` instead of `[(k1, v1), (k2, v2)]` (i.e. as a `map`).

— `n`: will format without the brackets. This will let you, for instance, format a range as `a, b, c` or `{a, b, c}` or `(a, b, c)` or however else you want, simply by providing the desired format string. If

printing a normal range, the brackets removed are `[]`. If printing as a map, the brackets removed are `{}`. If printing as a quoted string, the brackets removed are the `""`s (but escaping will still happen).

Additionally, ranges will support the same fill/align/width specifiers as in *std-format-spec*, for convenience and consistency.

If no element-specific formatter is provided (i.e. there is no inner colon - an empty element-specific formatter is still an element-specific formatter), the range will be formatted as debug. Otherwise, the element-specific formatter will be parsed and used.

To revisit a few rows from the earlier table:

Format String	Contents	Formatted Output
`{}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`['H', '\t', 'l', 'l', 'o']`
`{::}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`[H,  , l, l, o]`
`{::?c}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`['H', '\t', 'l', 'l', 'o']`
`{::d}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`[72, 9, 108, 108, 111]`
`{::#x}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`[0x48, 0x9, 0x6c, 0x6c, 0x6f]`
`{:s}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`H  llo`
`{:?s}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`"H\tllo"`
`{}`	`vector{vector{'a'}, vector{'b', 'c'}}`	`[['a'], ['b', 'c']]`
`{::?s}`	`vector{vector{'a'}, vector{'b', 'c'}}`	`["a", "bc"]`
`{:::d}`	`vector{vector{'a'}, vector{'b', 'c'}}`	`[[97], [98, 99]]`

The second row is not printed quoted, because an empty element specifier is provided. We assume that if the user explicitly provides a format specifier (even if it's empty), that they want control over what they're doing. The third row is printed quoted again because it was explicitly asked for using the `?c` specifier, applied to each character.

The last row, `:::d`, is parsed as:

	top level outer vector		top level inner vector		inner vector each element
`:`	(none)	`:`	(none)	`:`	`d`

That is, the `d` format specifier is applied to each underlying `char`, which causes them to be printed as integers instead of characters.

Note that you can provide both a fill/align/width specifier to the range itself as well as to each element:

Format String	Contents	Formatted Output
`{}`	`vector<int>{1, 2, 3}`	`[1, 2, 3]`
`{::*^5}`	`vector<int>{1, 2, 3}`	`[**1**, **2**, **3**]`
`{:o^17}`	`vector<int>{1, 2, 3}`	`oooo[1, 2, 3]oooo`
`{:o^29:*^5}`	`vector<int>{1, 2, 3}`	`oooo[**1**, **2**, **3**]oooo`

3.3.7.3 Pair and Tuple Specifiers

This is the hard part.

To start with, we for consistency will support the same fill/align/width specifiers as usual.

And likewise an `n` specifier to omit the parentheses and an `m` speciifer to format `pairs` and 2-`tuples` as `k: v` rather than `(k, v)`.

For ranges, we can have the underlying element's `formatter` simply parse the whole format specifier string from the character past the `:` to the `}`. The range doesn't care anymore at that point, and what we're left with is a specifier that the underlying element should understand (or not).

But for `pair`, it's not so easy, because format strings can contain *anything*. Absolutely anything. So when trying to parse a format specifier for a `pair<X, Y>`, how do you know where `X`'s format specifier ends and `Y`'s format specifier begins? This is, in general, impossible.

In [P2286R3], this paper used Tim's insight to take a page out of `sed`'s book and rely on the user providing the specifier string to actually know what they're doing, and thus provide their own delimiter. `pair` will recognize the first character that is not one of its formatters as the delimiter, and then delimit based on that. This previous revision had proposed the following:

Format String	Contents	Formatted Output
`{}`	`pair(10, 1729)`	`(10, 1729)`
`{:}`	`pair(10, 1729)`	`(10, 1729)`
`{::#x:04X}`	`pair(10, 1729)`	`(0xa, 06C1)`
`{:|#x|04X}`	`pair(10, 1729)`	`(0xa, 06C1)`
`{:Y#xY04X}`	`pair(10, 1729)`	`(0xa, 06C1)`

The last three rows are equivalent, the difference is which character is used to delimit the specifiers: `:` or `|` or `Y`.

This approach, while technically functional, still leaves something to be desired. For one thing, these examples are already difficult to read and I haven't even shown any additional nesting. We're using to nested parentheses, brackets, or braces, but there's nothing visually nested here. And it's not even clear

how to do something like that anyway. Several people expressed a desire to have a delimiter language that at least has some concept of nesting built-in - such as naturally-nesting punctuation like `()`s, `[]`s, or `{}`s (Unicode has plenty of other pairs of open/close characters. I could revisit my Russian roots with « and », or use something prettier like ⦑ and ⦒).

The point, ultimately, is that it is difficult to comme up with a format specifier syntax that works *at all* in the presence of types that can use arbitrary characters in their specifiers. Like formatting `std::chrono::system_clock::now()`:

Format String	Formatted Output
`{}`	`2021-10-24 20:33:37`
`{:%Y-%m-%d}`	`2021-10-24`
`{:%H:%M:%S}`	`20:33:37`
`{:%H hours, %M minutes, %S seconds}`	`20 hours, 33 minutes, 37 seconds`

Because there is reasonable concern about the complexity of the initially proposed solution, and because there doesn't seem to be a lot of demand for actually being able to do this, in contrast to the very clear and present demand of being able to format pairs and tuples simply by default - this revision of this paper is withdrawing this part of the proposal in an effort to get the rest of the paper in for C++23.

To summarize: `std::pair` and `std::tuple` will only support:

— the fill/align/width specifiers from *std-format-spec*

— the `?` specifier, to format as debug (which is a no-op, since it will always format as debug, since there is no opt-out provided)

— the `n` specifier, to omit the parentheses

— the `m` specifier, only valid for `pair` or 2-tuple, to format as `k: v` instead of `(k, v)`

For `tuple` of size other than 2, this will throw an exception (since you cannot format those as a map). To clarify the map specifier:

Format String	Contents	Formatted Output
`{}`	`pair(1, 2)`	`(1, 2)`
`{:m}`	`pair(1, 2)`	`1: 2`
`{:m}`	`tuple(1, 2)`	`1: 2`
`{}`	`tuple(1)`	`(1)`
`{:m}`	`tuple(1)`	exception or compile error
`{}`	`tuple(1, 2, "3"s)`	`(1, 2, "3")`
`{:m}`	`tuple(1, 2, "3"s)`	exception or compile error

§ 3.3.8 Escaping Behavior

There is some established practice for how to escape strings, for instance in Python and Rust, which seems like a really good idea to follow.

3.3.8.1 Python

In Python, the choice of characters to escape and the new algorithm for `repr` is described in [PEP-3138]:

> Characters that should be escaped are defined in the Unicode character database as:
>
> — Cc (Other, Control)
>
> — Cf (Other, Format)
>
> — Cs (Other, Surrogate)
>
> — Co (Other, Private Use)
>
> — Cn (Other, Not Assigned)
>
> — Zl (Separator, Line), refers to LINE SEPARATOR (`'\u2028'`).
>
> — Zp (Separator, Paragraph), refers to PARAGRAPH SEPARATOR (`'\u2029'`).
>
> — Zs (Separator, Space) other than ASCII space (`'\x20'`). Characters in this category should be escaped to avoid ambiguity.
>
> The algorithm to build `repr()` strings should be changed to:
>
> — Convert CR, LF, TAB and `'\'` to `'\r'`, `'\n'`, `'\t'`, `'\\'`.
>
> — Convert non-printable ASCII characters (0x00-0x1f, 0x7f) to `'\xXX'`.
>
> — Convert leading surrogate pair characters without trailing character (0xd800-0xdbff, but not followed by 0xdc00-0xdfff) to `'\uXXXX'`.
>
> — Convert non-printable characters ([… see above …]) to `'xXX'`, `'\uXXXX'` or `'\U00xxxxxx'`.
>
> — Backslash-escape quote characters (apostrophe, 0x27) and add a quote character at the beginning and the end.

3.3.8.2 Rust

Rust doesn't have (to my knowledge) such a formal description of which characters need to be escaped, I'm not sure if there's a Rust-equivalent of a PEP that I could link to. Rust's implementation gives a standard library function `is_printable` which is actually generated from a Python file which contains the following relevant logic:

```
def get_escaped(codepoints):
    for c in codepoints:
        if (c.class_ or "Cn") in "Cc Cf Cs Co Cn Zl Zp Zs".split() and
        c.value != ord(' '):
            yield c.value
```

Which is the exact same logic as Python: those eight classes, with the exception of ASCII space, are escaped. Looking at the actual Rust implementation code is a little bit more involved, but that's only because it's optimized for values and no longer based on actual structural elements from Unicode. Rust's actual algorithm for using this `is_printable` function can be found in the `impl Debug for str` found

here which is implemented in terms of `escape_debug_ext` (for clarity, in the context of printing a debug string, `args.escape_grapheme_extended` is `true`, `args.escape_single_quote` is `false`, and `args.escape_double_quote` is `true`. For a debug character, the latter two are flipped):

```
pub(crate) fn escape_debug_ext(self, args: EscapeDebugExtArgs) ->
        EscapeDebug {
    let init_state = match self {
        '\t' => EscapeDefaultState::Backslash('t'),
        '\r' => EscapeDefaultState::Backslash('r'),
        '\n' => EscapeDefaultState::Backslash('n'),
        '\\' => EscapeDefaultState::Backslash(self),
        '"' if args.escape_double_quote =>
        EscapeDefaultState::Backslash(self),
        '\'' if args.escape_single_quote =>
        EscapeDefaultState::Backslash(self),
        _ if args.escape_grapheme_extended && self.is_grapheme_extended() =>
        {
            EscapeDefaultState::Unicode(self.escape_unicode())
        }
        _ if is_printable(self) => EscapeDefaultState::Char(self),
        _ => EscapeDefaultState::Unicode(self.escape_unicode()),
    };
    EscapeDebug(EscapeDefault { state: init_state })
}
```

The grapheme-extended logic exists in Rust but not in Python, the rest is the same.
`char::escape_unicode` will:

> This will escape characters with the Rust syntax of the form `\u{NNNNNN}` where `NNNNNN` is a hexadecimal representation.

which, for example:

```
for c in '❤'.escape_unicode() {
    print!("{}", c);
}
println!();
```

will print `"\u{2764}"`. Though note that `println!("{:?}", "❤");` will just print that heart (quoted) because that heart is printable.

3.3.8.3 Golang

`golang`'s unicode package provides an `isPrint` function defined as follows:

```
func IsPrint(r rune) bool
```

> `IsPrint` reports whether the rune is defined as printable by Go. Such characters include letters, marks, numbers, punctuation, symbols, and the ASCII space character, from categories L, M, N, P, S and the ASCII space character. This categorization is the same as IsGraphic except that the only spacing character is ASCII space, `U+0020`.

In this case, Go is adding categories L, M, N, P, S, and ASCII space… whereas Rust and Python are removing categories Z and C but keeping ASCII space. These two sets are equivalent: the full set of Unicode category classes is L, M, N, P, S, Z, C. Hence, Go's logic is also the same as Rust and Python.

3.3.8.4 Proposed for C++

Escaping of a string in a Unicode encoding is done by translating each UCS scalar value, or a code unit if it is not a part of a valid UCS scalar value, in sequence (Note that all the backslashes are escaped here as well):

— If a UCS scalar value is one of `'\t'`, `'\r'`, `'\n'`, `'\\'` or `'"'`, it is replaced with `"\\t"`, `"\\r"`, `"\\n"`, `"\\\\"` and `"\\\""` respectively.

— Otherwise, if a UCS scalar value has a Unicode property Separator (Z) or Other (C) other than space, it is replaced with its universal character name escape sequence in the form `"\\u{`*`simple-hexadecimal-digit-sequence`*`}"` as proposed by [P2290R2], where *simple-hexadecimal-digit-sequence* is a hexadecimal representation of the UCS scalar value without leading zeros.

— Otherwise, if a UCS scalar value has a Unicode property Grapheme_Extend and there are no UCS scalar values preceding it in the string without this property, it is replaced with its universal character name escape sequence as above.

— Otherwise, a code unit that is not a part of a valid UCS scalar value is replaced with a hexadecimal escape sequence in the form `"\\x{`*`simple-hexadecimal-digit-sequence`*`}"` as proposed by [P2290R2], where *simple-hexadecimal-digit-sequence* is a hexadecimal representation of the code unit without leading zeros.

— Otherwise, a UCS scalar value is copied as is.

The same applies to wide strings with `'...'` and `"..."` replaced with `L'...'` and `L"..."` respectively.

For non-Unicode encodings an implementation-defined equivalent of Unicode properties is used.

Escape rules for characters are similar except that `'\''` is escaped instead of `'"'` and `'"'` is not escaped.

Examples:

```
std::cout << std::format("{:?}", std::string("h\tllo"));
// Output: "h\tllo"

std::cout << std::format("{:?}", std::string("\0 \n \t \x02 \x1b", 9));
// Output: "\u{0} \n \t \u{2} \u{1b}"

std::cout << std::format("{:?}, {:?}, {:?}", " \" ' ", '"', '\'');
// Output: " \" ' ", '"', '\''

std::cout << std::format("{:?}", "\xc3\x28"); // invalid UTF-8
// Output: "\x{c3}\x{28}"

std::cout << std::format("{:?}", "\u0300"); // assuming a Unicode encoding
// Output: "\u{300}"
// (as opposed to "̀" with an accent on the first ")
```

```
auto s = std::format("{:?}", "Привет, 🏴‍☠️!"); // assuming a Unicode encoding
// s == "\"Привет, 🏴‍☠️!\""
```

§ 3.4 Implementation Challenges

The previous revision of this paper ([P2286R4]) had a long section about the implementation challenges of this section, which existed to motivate the addition of two additional APIs to the standard: `retargeted_format_context` and `end_sentry`. However, since those APIs have been removed from the proposal (possibly to be included in a future, different paper), it doesn't make sense to have a long section about it in this particular paper.

For those curious, the previous text can be found here.

§ 3.5 How to support those views which are not `const`-iterable?

In a previous revision of this paper, this was a real problem since at the time `std::format` accepted its arguments by `const Args&...`

However, [P2418R2] was speedily adopted specifically to address this issue, and now `std::format` accepts its arguments by `Args&&...` This allows those views which are not `const`-iterable to be mutably passed into `format()` and `print()` and then mutably into its formatter. To support both `const` and non-`const` formatting of ranges without too much boilerplate, we can do it this way:

```
template <formattable V>
struct range_formatter {
    template <typename ParseContext>
    constexpr auto parse(ParseContext&);

    template <input_range R, typename FormatContext>
        requires same_as<remove_cvref_t<range_reference_t<R>>, V>
    constexpr auto format(R&&, FormatContext&);
};

template <input_range R> requires formattable<range_reference_t<R>>
struct formatter<R> : range_formatter<remove_cvref_t<range_reference_t<R>>>
{ };
```

`range_formatter` allows reducing unnecessary template instantiations. Any range of `int` is going to `parse` its specifiers the same way, there's no need to re-instantiate that code n times. Such a type will also help users to write their own formatters, since they can have a member `range_formatter<int>` to handle any range of `int` (or `int&` or `int const&`) rather than having to have a specific `formatter<my_special_range>`.

§ 3.6 Interface of the proposed solution

The proposed API for range formatting is:

```
template <input_range R, class charT>
    requires (not same_as<remove_cvref_t<range_reference_t<R>>, R>)
```

```
        and formattable<range_reference_t<R>, charT>
struct formatter<R, charT>
    : range_formatter<remove_cvref_t<range_reference_t<R>>, charT>
{ };
```

Where the public-facing API of `range_formatter` is:

```
template <class T, class charT = char>
    requires formattable<T, charT>
struct range_formatter {
    void set_separator(basic_string_view<charT>);
    void set_brackets(basic_string_view<charT>, basic_string_view<charT>);
    auto underlying() -> formatter<T, charT>&;

    template <typename ParseContext>
    constexpr auto parse(ParseContext&) -> ParseContext::iterator;

    template <typename R, typename FormatContext>
        requires same_as<T, remove_cvref_t<range_reference_t<R>>>
    auto format(R&&, FormatContext&) const -> FormatContext::iterator;
};
```

The reason for this shape, rather than putting all the implementation directly into the particular specialization of `formatter<R, charT>`, is that it makes it much easier to implement custom formatting for other ranges. You can see an example in the implementation of `format_join` in the next section. Or, even simpler, implementing formatting for `std::map` and `std::set`:

```
template <formattable Key, formattable T, class Compare, class Allocator>
struct formatter<map<Key, T, Compare, Allocator>>
    : range_formatter<pair<Key const, T>>
{
    formatter() {
        this->set_brackets("{", "}");
        this->underlying().set_brackets({}, {});
        this->underlying().set_separator(": ");
    }
};

template <formattable Key, class Compare, class Allocator>
struct formatter<set<Key, Compare, Allocator>>
    : range_formatter<Key>
{
    formatter() {
        this->set_brackets("{", "}");
    }
};
```

However, this is only the case for ranges (where the user might actually need to implement formatting for their own range) and is not the case for pair and tuple. This is because the `pair` and `tuple` formatters aren't constrained on `tuple_like`. We don't even have such a concept. Those formatters are specific to `pair` and `tuple`. If we ever do add a `tuple_like` concept, at that point we can add a `tuple_formatter`.

The proposed API for pair and tuple formatting is (substituting `pair` and `tuple` in for *`TEMPLATE`*):

```
template <class charT, formattable<charT>... Ts>
struct formatter<TEMPLATE<Ts...>, charT>
{
    void set_separator(basic_string_view<charT>);
    void set_brackets(basic_string_view<charT>, basic_string_view<charT>);

    template <typename ParseContext>
    constexpr auto parse(ParseContext&) -> ParseContext::iterator;

    template <typename FormatContext>
    auto format(POSSIBLY-CONST& elems, FormatContext&) const ->
        FormatContext::iterator;
};
```

The type *POSSIBLY-CONST* is *TEMPLATE*`<Ts...> const` when that type is formattable (i.e. all of `Ts const...` are formattable) and *TEMPLATE*`<Ts...>` otherwise, in an effort to reduce unnecessary template instantiations.

Otherwise, it's a similar structure to `range_formatter` for similar reasons (except no `underlying()` since I'm not sure you need it).

§ 3.7 What additional functionality?

There's three layers of potential functionality:

1. Top-level printing of ranges: this is `fmt::print("{}", r);`

2. A format-joiner which allows providing a a custom delimiter: this is provided in `{fmt}` under the spelling `fmt::print("{:02x}", fmt::join(r, ":"))`. Previous revisions of the paper either sought to simply standardize this under the name `std::format_join` ([P2286R3]), or to add the ability to specify a custom delimiter under the `d` specifier ([P2286R4]), but this paper does not actually provide such a facility directly.

3. A more involved version of a format-joiner which takes a delimiter and a callback that gets invoked on each element. fmt does not provide such a mechanism, though the Rust itertools library does:

```
let matrix = [[1., 2., 3.],
              [4., 5., 6.]];
let matrix_formatter = matrix.iter().format_with("\n", |row, f| {
                                f(&row.iter().format_with(", ", |elt, g|
        g(&elt)))
                            });
assert_eq!(format!("{}", matrix_formatter),
           "1, 2, 3\n4, 5, 6");
```

The paper provides the tools to implement to implement (2) and (3), but does not directly propose either.

For example, here is an implementation of `format_join(r, delim)`:

```
template <std::ranges::input_range V>
    requires std::ranges::view<V>
          && std::formattable<std::ranges::range_reference_t<V>>
```

```cpp
struct format_join_view {
    V v;
    std::string_view delim;
};

template <typename V>
struct std::formatter<format_join_view<V>>
{
    std::range_formatter<std::remove_cvref_t<std::ranges::range_reference_t<V>>>
        underlying;

    template <typename ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        return underlying.parse(ctx);
    }

    template <typename R, typename FormatContext>
    auto format(R&& r, FormatContext& ctx) const {
        underlying.set_separator(r.delim);
        return underlying.format(r, ctx);
    }
};

template <std::ranges::viewable_range R>
    requires std::formattable<std::ranges::range_reference_t<R>>
auto format_join(R&& r, std::string_view delim) {
    return format_join_view{std::views::all(std::forward<R>(r)), delim};
}
```

§ 3.8 `format` or `std::cout`?

Just `format` is sufficient.

§ 3.9 What about `vector<bool>`?

Nobody expected this section.

The `value_type` of this range is `bool`, which is formattable. But the `reference` type of this range is `vector<bool>::reference`, which is not. In order to make the whole type formattable, we can either make `vector<bool>::reference` formattable (and thus, in general, a range is formattable if its `reference` types is formattable) or allow formatting to fall back to constructing a `value_type` for each `reference` (and thus, in general, a range is formattable if either its `reference` type or its `value_type` is formattable).

For most ranges, the `value_type` is `remove_cvref_t<reference>`, so there's no distinction here between the two options. And even for `zip` [P2321R2], there's still not much distinction since it just wraps this question in tuple since again for most ranges the types will be something like `tuple<T, U>` vs `tuple<T&, U const&>`, so again there isn't much distinction.

`vector<bool>` is one of the very few ranges in which the two types are truly quite different. So it doesn't offer much in the way of a good example here, since `bool` is cheaply constructible from

`vector<bool>::reference`. Though it's also very cheap to provide a formatter specialization for `vector<bool>::reference`.

Rather than having the library provide a default fallback that lifts all the `reference` types to `value_types`, which may be arbitrarily expensive for unknown ranges, this paper proposes a format specialization for `vector<bool>::reference`. This type is actually defined as `vector<bool, Alloc>::reference`, so the wording for this aspect will be a little awkward (we'll need to provide a type trait *`is-vector-bool-reference`*`<R>`, etc., but this is a problem for the wording and the implementation to deal with).

§ 3.10 What about container adaptors?

The standard library has three container adaptors: `queue`, `priority_queue`, and `stack`. None of these are actually ranges, none of them defines a `begin()` or an `end()` or any kind of iterator. But they do all adapt a range, which is a specified protected member. It is still useful, especially for debugging purposes, to be able to simply print what's in your `stack`.

Note that we don't have to *specifically* add support for this, as users can always work around it themselves:

```
struct hack : std::stack<int> {
    using std::stack<int>::c;
};

int main() {
    std::stack<int> s;
    s.push(1);
    s.push(2);
    std::print("{}\n", s.*&hack::c);
}
```

That's valid, probably the best way to solve this problem, yet also not the kind of thing we want to encourage people to do. This paper thus proposes that `queue`, `priority_queue`, and `stack` are formattable as their underlying container type.

This does lead to one quirk, which is `priority_queue`. If we simply defer to the underlying container's formatting, then we get behavior like this:

```
int main() {
    std::priority_queue<int> s;
    for (int i = 0; i < 10; ++i) {
        s.push(i);
    }
    std::print("{}\n", s); // prints [9, 8, 5, 6, 7, 1, 4, 0, 3, 2]
}
```

That is not the order of elements in the `s`, at least not the way we typically think of things. `s.top()` is `9`, but the rest of the elements are not in this order. But also… that's fine. This is still a useful representation for formatting (this is exactly the underlying representation), they are free to either access `s.&hack::c`

and figure out how to print it in “the right order” or write their own `priority_queue` with its own custom formatting.

§ 3.11 Examples with user-defined types

Let’s say a user has a type like:

```
struct Foo {
    int bar;
    std::string baz;
};
```

And want to format `Foo{.bar=10, .baz="Hello World"}` as the string `Foo(bar=10, baz="Hello World")`. They can do so this way:

```
template <>
struct formatter<Foo, char> {
    template <typename FormatContext>
    constexpr auto format(Foo const& f, FormatContext& ctx) const {
        return format_to(ctx.out(), "Foo(bar={}, baz={:?})", f.bar, f.baz);
    }
};
```

How about wrappers?

Let’s say you have your own implementation of `Optional`, that you want to format the same way that Rust does: so that a disengaged one formats as `None` and an engaged one formats as `Some(??)`. We can start by:

```
template <formattable<char> T>
struct formatter<Optional<T>, char> {
    // we'll skip parse for now

    template <typename FormatContext>
    auto format(Optional<T> const& opt, FormatContext& ctx) {
        if (not opt) {
            return format_to(ctx.out(), "None");
        } else {
            return format_to(ctx.out(), "Some({})", *opt);
        }
    }
};
```

If we had an `Optional<string>("hello")`, this would format as `Some(hello)`. Which may be fine. But what if we wanted to format it as `Some("hello")` instead? That is, take advantage of the quoting rules described earlier. What do you write instead of `*opt` to format `string`s (or `char`s or user-defined string-like types) as quoted in this context?

We can both add support for quoting/escaping and also arbitrary specifiers at the same time:

```
template <formattable<char> T>
struct formatter<Optional<T>, char> {
```

```
    formatter<T, char> underlying;

    formatter() {
        if constexpr (requires { underlying.set_debug_format(); }) {
            underlying.set_debug_format();
        }
    }

    template <typenaem ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        return underlying.parse(ctx);
    }

    template <typename FormatContext>
    auto format(Optional<T> const& opt, FormatContext& ctx) {
        if (not opt) {
            return format_to(ctx.out(), "None");
        } else {
            ctx.advance_to(format_to(ctx.out(), "Some("));
            auto out = underlying.format(*opt, ctx);
            *out++ = ')';
            return out;
        }
    }
};
```

This lets me format `Optional<string>("hello")` as `Some("hello")` by default, or format `Optional<int>(42)` as `Some(0x2a)` if I provide the specifier string `"{:#x}"`.

§ 4 Proposal

The standard library will provide the following utilities:

— A `formattable` concept.

— A `range_formatter<V>` that uses a `formatter<V>` to `parse` and `format` a range whose `reference` is similar to `V`. This can accept a specifier on the range (align/pad/width as well as string/map/debug/empty) and on the underlying element (which will be applied to every element in the range). This will additionally have a few public member functions to facilitate users build custom range formatters, as detailed here:

  — `set_separator(string_view)`

  — `set_brackets(string_view, string_view)`

  — `underlying()`

The standard library should add specializations of `formatter` for:

— any type `R` that is an `input_range` whose `reference` is `formattable`, which inherits from `range_formatter<remove_cvref_t<ranges::range_reference_t<R>>>`

— `pair<T, U>` if `T` and `U` are `formattable` (additionally with `set_separator` and `set_brackets`)

— `tuple<Ts...>` if all of `Ts...` are `formattable` (additionally with `set_separator` and `set_brackets`)

Additionally, the standard library should provide the following more specific specializations of `formatter`:

— `vector<bool, Alloc>::reference` (which formats as a `bool`)

— all the associative maps (`map`, `multimap`, `unordered_map`, `unordered_multimap`) if their respective key/value types are `formattable`. This accepts the same set of specifiers as any other range, except by *default* it will format as `{k: v, k: v}` instead of `[(k, v), (k, v)]`

— all the associative sets (`sets`, `multiset`, `unordered_set`, `unordered_multiset`) if their respective key/value types are `formattable`. This accepts the same set of specifiers as any other range, except by *default* it will format as `{v1, v2}` instead of `[v1, v2]`

— `queue`, `stack`, and `priority_queue`, which defer to their underlying representations.

Formatting for `string`, `string_view`, `const char*`, and `char` (and all the `wchar_t` equivalents) will gain a `?` specifier as well as a `set_debug_format()` member function, which causes these types to be printed as escaped and quoted if provided. Ranges and tuples will, by default, print their elements as escaped and quoted, unless the user provides a specifier for the element.

§ 5 Wording

The wording here is grouped by functionality added rather than linearly going through the standard text.

§ 5.1 Concept `formattable`

First, we need to define a user-facing concept. We need this because we need to constrain `formatter` specializations on whether the underlying elements of the `pair`/`tuple`/range are formattable, and users would need to do the same kind of thing for their types. This is tricky since formatting involves so many different types, so this concept will never be perfect, so instead we're trying to be good enough.

Change 20.20.1 [format.syn]:

```
namespace std {
  // ...
  // [format.formatter], formatter
  template<class T, class charT = char> struct formatter;

  // [format.parse.ctx], class template basic_format_parse_context
  template<class charT> class basic_format_parse_context;
  using format_parse_context = basic_format_parse_context<char>;
  using wformat_parse_context = basic_format_parse_context<wchar_t>;
+ // [format.formattable], formattable
+ template<class T, class charT>
+   concept formattable = see below;
  // ...
}
```

Add a clause [format.formattable] under 20.20.6 [format.formatter] and likely after 20.20.6.1 [formatter.requirements]:

1 Let *fmt-iter-for*<charT> be an implementation-defined type that models output_iterator<const charT&> ([iterator.concept.output]).

```
template<class T, class charT>
concept formattable =
    semiregular<formatter<remove_cvref_t<T>, charT>> &&
    requires (formatter<remove_cvref_t<T>, charT> f,
              const formatter<remove_cvref_t<T>, charT> cf,
              T t,
              basic_format_context<fmt-iter-for<charT>, charT> fc,
              basic_format_parse_context<charT> pc) {
        { f.parse(pc) } ->
         same_as<basic_format_parse_context<charT>::iterator>;
        { cf.format(t, fc) } -> same_as<fmt-iter-for<charT>>;
    };
```

2 A type T and a character type charT model formattable if formatter<remove_cvref_t<T>, charT> meets the *BasicFormatter* requirements ([formatter.requirements]) and, if remove_reference_t<T> is const-qualified, the *Formatter* requirements.

§ 5.2 Additional formatting support for characters and strings

Change 20.20.2.2 [format.string.std] to add ? as a valid type:

> The syntax of format specifications is as follows:
>
> ```
> type: one of
>   a A b B c d e E f F g G o p s x X ?
> ```

Add ? to the strings table in 20.20.2.2 [format.string.std]/17 (Table 64):

17 The available string presentation types are specified in Table 64.

Type	Meaning
none, s	Copies the string to the output.
?	Copies the escaped string ([format.string.escaped]) to the output.

Add ? to the charT table in 20.20.2.2 [format.string.std]/20 (Table 66):

20 The available charT presentation types are specified in Table 66.

Type	Meaning
none, c	Copies the character to the output.
b,B,d,o,x,X	As specified in Table 65.
?	Copies the escaped character ([format.string.escaped]) to the output.

Add `set_debug_format()` to the character and string specializations in 20.20.6.2 [format.formatter.spec]:

1 The functions defined in [format.functions] use specializations of the class template `formatter` to format individual arguments.

2 Let `charT` be either `char` or `wchar_t`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, non-static member function `set_debug_format()` which will cause these formatters to interpret the format specification as if its *type* were `?`. Each header that declares the template `formatter` provides the following enabled specializations:

(2.1) — The debug-enabled specializations

```
template<> struct formatter<char, char>;
template<> struct formatter<char, wchar_t>;
template<> struct formatter<wchar_t, wchar_t>;
```

(2.2) — For each `charT`, the debug-enabled string type specializations

```
template<> struct formatter<charT*, charT>;
template<> struct formatter<const charT*, charT>;
template<size_t N> struct formatter<const charT[N], charT>;
template<class traits, class Allocator>
  struct formatter<basic_string<charT, traits, Allocator>, charT>;
template<class traits>
  struct formatter<basic_string_view<charT, traits>, charT>;
```

(2.3) — For each `charT`, for each *cv*-unqualified arithmetic type `ArithmeticT` other than `char`, `wchar_t`, `char8_t`, `char16_t`, or `char32_t`, a specialization

```
template<> struct formatter<ArithmeticT, charT>;
```

(2.4) — For each `charT`, the pointer type specializations

```
template<> struct formatter<nullptr_t, charT>;
template<> struct formatter<void*, charT>;
template<> struct formatter<const void*, charT>;
```

Add a new clause [format.string.escaped] “Formatting escaped characters and strings” which will discuss what it means to do escaping.

1 A character or string can be formatted as *escaped* to make it more suitable for debugging or for logging.

2 The escaped string representation of a string, *s*, in a Unicode encoding consists of the following sequence of scalar values:

(2.1) — A U+0022 QUOTATION MARK (") character

(2.2) — For each UCS scalar value in *s*, or a code unit if it is not a part of a valid UCS scalar value:

(2.3) — If the UCS scalar value is in the table below, then its corresponding two-character escape sequence:

UCS scalar value	escape sequence
U+0009 CHARACTER TABULATION	`\t`
U+000A LINE FEED	`\n`
U+000D CARRIAGE RETURN	`\r`
U+0022 QUOTATION MARK	`\"`
U+005C REVERSE SOLIDUS	`\\`

(2.4) — Otherwise, if the UCS scalar value

(2.4.1) — is not U+0020 SPACE and has the Unicode property `General_Category=Separator` (`Z`) or `General_Category=Other` (`C`), or

(2.4.2) — has the Unicode property `Grapheme_Extend=Yes`

then its universal character name escape sequence in the form `\u{`*`simple-hexadecimal-digit-sequence`*`}`, where *`simple-hexadecimal-digit-sequence`* is the shortest hexadecimal representation of the UCS scalar value using lower-case *`hexadecimal-digit`*s.

(2.5) — Otherwise, if it is a code unit that is not a part of a valid UCS scalar value, then a hexadecimal escape sequence in the form `\x{`*`simple-hexadecimal-digit-sequence`*`}`, where *`simple-hexadecimal-digit-sequence`* is the shortest hexadecimal representation of the code unit using lower-case *`hexadecimal-digit`*s.

(2.6) — Otherwise, the UCS scalar value as-is.

(2.7) — Finally, another U+0022 QUOTATION MARK (`"`) character.

3 The escaped character representation of a character, *`c`*, in a Unicode encoding is equivalent to the escaped string representation a string of *`c`*, except that the result starts and ends with U+0027 APOSTROPHE (`'`) instead of U+0022 QUOTATION MARK (`"`) and U+0027 APOSTROPHE is escaped as `\'` while U+0022 QUOTATION MARK is left unchanged.

4 The escaped character and escaped string representations of a character or string in a non-Unicode encoding is implementation-defined.

[*Example*:

```
string s0 = format("[{}]", "h\tllo");                  // s0 has value: [h
        llo]
string s1 = format("[{:?}]", "h\tllo");                // s1 has value:
        ["h\tllo"]
string s2 = format("[{:?}]", "Спасибо, Виктор ♥!");    // s2 has value:
        ["Спасибо, Виктор ♥!"]
string s3 = format("[{:?}] [{:?}]", '\'', '"');        // s3 has value:
        ['\'', '"']
```

```
string s4 = format("[{:?}]", string("\0 \n \t \x02 \x1b", 9));
                                                    // s4 has value
        [\u{0} \n \t \u{2} \u{1b}]
string s5 = format("[{:?}]", "\xc3\x28");           // invalid UTF-8
                                                    // s5 has value:
        ["\x{c3}\x{28}"]
string s6 = format("[{:?}]", "🤦‍♂️");                  // s6 has value: ["🤦
        \u{200d}♂\u{fe0f}"]
```

-end example]

§ 5.3 Formatting for ranges

Add to 20.20.1 [format.syn]:

```
namespace std {
  // ...

  // [format.formatter], formatter
  template<class T, class charT = char> struct formatter;

+ // [format.range], range formatter
+ template<class T, class charT = char>
+     requires formattable<T, charT>
+   class range_formatter;
+
+ template<ranges::input_range R, class charT>
+         requires (not
        same_as<remove_cvref_t<ranges::range_reference_t<R>>, R>)
+           && formattable<ranges::range_reference_t<R>, charT>
+   struct formatter<R, charT>
+     : range_formatter<remove_cvref_t<ranges::range_reference_t<R>>, charT>
+   { };

  // ...
}
```

And a new clause [format.range]:

1 The class template `range_formatter` is a convenient utility for implementing `formatter` specializations for range types.

2 `range_formatter` interprets *`format-spec`* as a *`range-format-spec`*. The syntax of format specifications is as follows:

`range-format-spec`:
    `range-fill-and-align`$_{opt}$ *`width`*$_{opt}$ `n`$_{opt}$ *`range-type`*$_{opt}$ *`range-underlying-spec`*$_{opt}$

`range-fill-and-align`:
    `range-fill`$_{opt}$ *`align`*

`range-fill`:
    any character other than { or } or :

```
range-type:
    m
    s
    ?s

range-underlying-spec:
    : format-spec
```

3 For `range_formatter<T, charT>`, the *`format-spec`* in a *`range-underlying-spec`*, if any, is interpreted by `formatter<T, charT>`.

4 The *`range-fill-and-align`* is interpreted the same way as a *`fill-and-align`* ([format.string.std]). The productions *`align`* and *`width`* are described in [format.string].

5 The `n` option causes the range to be formatted without the open and close brackets. [*Note*: this is equivalent to invoking `set_brackets({}, {})` - *end note*]

6 The *`range-type`* specifier changes the way a range is formatted, with certain options only valid with certain argument types. The meaning of the various type options is as specified in Table X.

Option	Requirements	Meaning
`m`	`T` shall be either a specialization of `pair` or a specialization of `tuple` such that `tuple_size<T>::value` is `2`	Indicates that the open bracket should be `"{"`, the close bracket should be `"}"`, the separator should be `", "`, and each range element should be formatted as if `m` were specified for its *`tuple-type`*. [*Note*: if the `n` option is also provided, both the open and close brackets are still empty. - *end note*]
`s`	`T` shall be `charT`	Indicates that the range should be formatted as a `string`.
`?s`	`T` shall be `charT`	Indicates that the range should be formatted as an escaped `string` ([format.string.escaped]).

If the *`range-type`* is `s` or `?s`, then there shall be no `n` option and no *`range-underlying-spec`*.

```
namespace std {
  template<class T, class charT = char>
    requires formattable<T, charT>
  class range_formatter {
    formatter<T, charT> underlying_;
        // exposition only
    basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(", ");
        // exposition only
    basic_string_view<charT> open-bracket_ = STATICALLY-WIDEN<charT>("[");
        // exposition only
    basic_string_view<charT> close-bracket_ = STATICALLY-WIDEN<charT>("]");
        // exposition only

  public:
    range_formatter() = default;

    void set_separator(basic_string_view<charT> sep);
    void set_brackets(basic_string_view<charT> open,
        basic_string_view<charT> close);
    formatter<T, charT>& underlying() { return underlying_; }

    template <class ParseContext>
      constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

    template <ranges::input_range R, class FormatContext>
        requires same_as<remove_cvref_t<ranges::range_reference_t<R>>, T>
      typename FormatContext::iterator
        format(R&& r, FormatContext& ctx);
  };
}

void set_separator(basic_string_view<charT> sep);
```

7 *Effects*: Equivalent to `separator_ = sep;`

```
void set_brackets(basic_string_view<charT> open, basic_string_view<charT>
        close);
```

8 *Effects*: Equivalent to

> ```
> open-bracket_ = open;
> close-bracket_ = close;
> ```

```
template <class ParseContext>
  constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

9 *Effects*: Parses the format specifier as a *range-format-spec* and stores the parsed specifiers in `*this`. Unless the *range-type* is either `s` or `?s`, if *underlying_*`.set_debug_format()` is a valid expression and there is no *range-underlying-spec*, calls *underlying_*`.set_debug_format()`.

10 *Returns*: an iterator past the end of the *range-format-spec*.

```
template <ranges::input_range R, class FormatContext>
    requires same_as<remove_cvref_t<ranges::range_reference_t<R>>, T>
  typename FormatContext::iterator
    format(R&& r, FormatContext& ctx);
```

11 *Effects*: Writes the following into `ctx.out()`, adjusted according to the *`range-format-spec`*:

(11.1) — If the *`range-type`* was `s`, then as if by formatting `basic_string<charT>(from_range, r)`.

(11.2) — Otherwise, if the *`range-type`* was `?s`, then as if by formatting `basic_string<charT>(from_range, r)` as an escaped string ([format.string.escaped]).

(11.3) — Otherwise,

(11.3.1) — *`open-bracket_`*

(11.3.2) — for each element, `e`, of the range `r`:

(11.3.2.1) — the result of writing `e` via *`underlying_`*

(11.3.2.2) — *`separator_`*, unless `e` is the last element of `r`

(11.3.3) — *`close-bracket_`*

12 *Returns*: an iterator past the end of the output range.

§ 5.3.1 Formatting for specific ranges: all the maps and sets

Add a clause (maybe after 22.5 [unord] and before 22.6 [container.adaptors]) [assoc.format] Associative Formatting:

1 For each of `map`, `multimap`, `unordered_map`, and `unordered_multimap`, the library provides the following formatter specialization where *`map-type`* is the name of the template:

```
namespace std {
  template <class charT, formattable<charT> Key, formattable<charT> T,
        class... U>
  struct formatter<map-type<Key, T, U...>, charT> :
        range_formatter<pair<const Key, T>>
  {
    formatter();
  };
}

formatter();
```

2 *Effects*: Equivalent to:

```
this->set_brackets(STATICALLY-WIDEN<charT>("{"), STATICALLY-WIDEN<charT>
        ("}"));
this->underlying().set_brackets({}, {});
this->underlying().set_separator(STATICALLY-WIDEN<charT>(": "));
```

3 For each of `set`, `multiset`, `unordered_set`, and `unordered_multiset`, the library provides the following formatter specialization where *`set-type`* is the name of the template:

```
namespace std {
  template <class charT, formattable<charT> Key, class... U>
  struct formatter<set-type<Key, U...>, charT> : range_formatter<Key, charT>
  {
    formatter();
  };
}

formatter();
```

4 *Effects*: Equivalent to:

```
this->set_brackets(STATICALLY-WIDEN<charT>("{"), STATICALLY-WIDEN<charT>
        ("}"));
```

§ 5.3.2 Formatting for specific ranges: all the container adaptors

At the end of 22.6 [container.adaptors], add a clause [container.adaptors.format]:

1 For each of `queue`, `priority_queue`, and `stack`, the library provides the following formatter specialization where *`adaptor-type`* is the name of the template:

```
namespace std {
  template <class charT, class T, formattable<charT> Container, class... U>
  struct formatter<adaptor-type<T, Container, U...>, charT>
  {
  private:
    formatter<Container, charT> underlying_; // exposition only

  public:
    template <class ParseContext>
      constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

    template <class FormatContext>
      typename FormatContext::iterator
        format(const adaptor-type<T, Container, U...>& r, FormatContext&
        ctx) const;
  };
}

template <class ParseContext>
  constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

2 *Effects*: Equivalent to `return underlying_.parse(ctx);`

```
template <class FormatContext>
  typename FormatContext::iterator
    format(const adaptor-type<T, Container, U...>& r, FormatContext& ctx)
        const;
```

3 *Effects*: Equivalent to `return underlying_.format(r.c, ctx);`

§ 5.4 Formatting for `pair` and `tuple`

And a new clause [format.tuple]:

1 For each of `pair` and `tuple`, the library provides the following formatter specialization where *`tuple-type`* is the name of the template:

```
namespace std {
template <class charT, formattable<charT>... Ts>
  struct formatter<tuple-type<Ts...>, charT> {
  private:
    tuple<formatter<remove_cvref_t<Ts>, charT>...> underlying_;
        // exposition only
    basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(", ");
        // exposition only
    basic_string_view<charT> open-bracket_ = STATICALLY-WIDEN<charT>("(");
        // exposition only
    basic_string_view<charT> close-bracket_ = STATICALLY-WIDEN<charT>(")");
        // exposition only

  public:
    void set_separator(basic_string_view<charT> sep);
    void set_brackets(basic_string_view<charT> open,
        basic_string_view<charT> close);

    template <class ParseContext>
      constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

    template <class FormatContext>
      typename FormatContext::iterator
        format(see below& elems, FormatContext& ctx) const;
  };
}
```

2 The `parse` member functions of these formatters interpret the format specification as a *`tuple-format-spec`* according to the following syntax:

`tuple-format-spec`:
    `tuple-fill-and-align`$_{\texttt{opt}}$ *`width`*$_{\texttt{opt}}$ *`tuple-type`*$_{\texttt{opt}}$

`tuple-fill-and-align`:
    `tuple-fill`$_{\texttt{opt}}$ *`align`*

`tuple-fill`:
    any character other than { or } or :

`tuple-type`:
    m
    n

3 The *`tuple-fill-and-align`* is interpreted the same way as a *`tuple-and-align`* ([format.string.std]). The productions *`align`* and *`width`* are described in [format.string].

4 The *tuple-type* specifier changes the way a `pair` or `tuple` is formatted, with certain options only valid with certain argument types. The meaning of the various type options is as specified in Table X.

Option	Requirements	Meaning
`m`	`sizeof...(Ts) == 2`	Indicates that the open and close bracket should be `""` and the separator should be `": "`.
`n`	None	Indicates that the open and close bracket should be `""`.

```
void set_separator(basic_string_view<charT> sep);
```

5 *Effects*: Equivalent to *separator_* `= sep;`

```
void set_brackets(basic_string_view<charT> open, basic_string_view<charT>
        close);
```

6 *Effects*: Equivalent to

> *open-bracket_* `= open;`
> *close-bracket_* `= close;`

```
template <class ParseContext>
  constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

7 *Effects*: Parses the format specifier as a *tuple-format-spec* and stores the parsed specifiers in `*this`. For each element *e* in *underlying_*, if *e*`.set_debug_format()` is a valid expression, calls *e*`.set_debug_format()`.

8 *Returns*: an iterator past the end of the *tuple-format-spec*.

```
template <class FormatContext>
  typename FormatContext::iterator
    format(see below& elems, FormatContext& ctx) const;
```

9 The type of `elems` is:

(9.1) — If `(formattable<const Ts, charT> && ...)` is `true`, `const` *tuple-type*`<Ts...>&`.

(9.2) — Otherwise *tuple-type*`<Ts...>&`.

10 *Effects*: Writes the following into `ctx.out()`, adjusted according to the *tuple-format-spec*:

(10.1) — *open-bracket_*

(10.2) — for each index `I` from `0` up to `sizeof...(Ts)`, exclusive:

(10.2.1) — if `I != 0`, *separator_*

(10.2.2) — the result of writing `get<I>(elems)` via `get<I>(underlying_)`

(10.3) — *`close-bracket_`*

11 *Returns*: an iterator past the end of the output range.

§ 5.5 Formatter for `vector<bool>::reference`

Add to 22.3.6 [vector.syn]

```
namespace std {
  // [vector], class template vector
  template<class T, class Allocator = allocator<T>> class vector;

  // ...

  // [vector.bool], class vector<bool>
  template<class Allocator> class vector<bool, Allocator>;

+ template<class T>
+   inline constexpr bool is-vector-bool-reference = see below; //
        exposition only

+ template<class T, class charT> requires is-vector-bool-reference<T>
+   struct formatter<T, charT>;
```

Add to [vector.bool] at the end:

```
template<class R>
  inline constexpr bool is-vector-bool-reference = see below;
```

8 The variable template *`is-vector-bool-reference`*`<T>` is `true` if `T` denotes the type `vector<bool, Alloc>::reference` for some type `Alloc` and `vector<bool, Alloc>` is not a program-defined specialization.

```
template<class T, class charT> requires is-vector-bool-reference<T>
  struct formatter<T, charT> {
  private:
    formatter<bool, charT> fmt;     // exposition only

  public:
    template <class ParseContext>
      constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

    template <class FormatContext>
      typename FormatContext::iterator
        format(const T& ref, FormatContext& ctx) const;
  };

template <class ParseContext>
  constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

9 *Effects*: Equivalent to `return fmt.parse(ctx);`

```
template <class FormatContext>
  typename FormatContext::iterator
    format(const T& ref, FormatContext& ctx) const;
```

10 *Effects*: Equivalent to `return fmt.format(ref, ctx);`

6 Acknowledgments

Thanks to Victor Zverovich for `{fmt}`, explanation of Unicode, and numerous design discussions. Thanks to Peter Dimov for design feedback. Thanks to Tim Song for invaluable help on the design and wording.

7 References

[LWG3478] Barry Revzin. views::split drops trailing empty range.
https://wg21.link/lwg3478

[LWG3636] Arthur O'Dwyer. formatter::format should be const-qualified.
https://wg21.link/lwg3636

[P1636R2] Lars Gullik Bjønnes. 2019-10-06. Formatters for library types.
https://wg21.link/p1636r2

[P2214R1] Barry Revzin, Conor Hoekstra, Tim Song. 2021-09-14. A Plan for C++23 Ranges.
https://wg21.link/p2214r1

[P2286R0] Barry Revzin. 2021-01-15. Formatting Ranges.
https://wg21.link/p2286r0

[P2286R1] Barry Revzin. 2021-02-19. Formatting Ranges.
https://wg21.link/p2286r1

[P2286R2] Barry Revzin. 2021-08-16. Formatting Ranges.
https://wg21.link/p2286r2

[P2286R3] Barry Revzin. 2021-11-17. Formatting Ranges.
https://wg21.link/p2286r3

[P2286R4] Barry Revzin. 2021-12-18. Formatting Ranges.
https://wg21.link/p2286r4

[P2290R2] Corentin Jabot. 2021-07-15. Delimited escape sequences.
https://wg21.link/p2290r2

[P2321R2] Tim Song. 2021-06-11. zip.
https://wg21.link/p2321r2

[P2418R0] Victor Zverovich. 2021-08-08. Add support for std::generator-like types to std::format. https://wg21.link/p2418r0

[P2418R2] Victor Zverovich. 2021-09-24. Add support for std::generator-like types to std::format. https://wg21.link/p2418r2

[PEP-3138] Atsuo Ishimoto. 2008. PEP 3138 – String representation in Python 3000. https://www.python.org/dev/peps/pep-3138/